```
#23070521013 Amritansh Singh

# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinM
from tensorflow.keras.models import Se
from tensorflow.keras.layers import LS
```

```
import re
from google.colab import files

print("Please upload your historical stock data CSV file.")
uploaded = files.upload()

if not uploaded:
    raise ValueError("No file was uploaded. Please upload a CSV file to proceed.")

# Get the filename of the uploaded file
file_name = list(uploaded.keys())[0]
print(f"Uploaded file: {file_name}")

# Load your actual stock data from the uploaded CSV file
df = pd.read_csv(file_name)

# Standardize column names and set 'Price'
# Assuming 'Date' is present and 'Close/Last' contains the relevant price data
if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'])
    df.set_index('Date', inplace=True)

# Identify and clean the 'Close/Last' column as 'Price'
if 'Close/Last' in df.columns:
    # Remove '$' and ',' from 'Close/Last' and convert to float
    df['Price'] = df['Close/Last'].astype(str).str.replace(r'[$,]', '', regex=True).astype(float)
    df = df[['Price']].copy() # Keep only the 'Price' column for consistency
elif 'Price' not in df.columns:
    print("Warning: Neither 'Close/Last' nor 'Price' column found. Please check your CSV and adjust the cod
    print("Attempting to use the first numeric column found as 'Price'...")
    numeric_cols = df.select_dtypes(include=np.number).columns
    if len(numeric_cols) > 0:
        # Assuming the first numeric column after Date processing is the relevant price
        df.rename(columns={numeric_cols[0]: 'Price'}, inplace=True)
        df = df[['Price']].copy()
    else:
        raise ValueError("No suitable numeric column found to use as 'Price'.")

# Drop any rows with NaN values in the 'Price' column that might result from conversion issues
df.dropna(subset=['Price'], inplace=True)

print("Loaded stock data:")
display(df.head())
display(df.tail())
```



```
Please upload your historical stock data CSV file.
Choose Files HistoricalD…9716990.csv
HistoricalData_1787559716990.csv(text/csv) - 1163 bytes, last modified: 8/24/2026 - 100% done
Saving HistoricalData_1787559716990.csv to HistoricalData_1787559716990.csv
Uploaded file: HistoricalData_1787559716990.csv
Loaded stock data:
```

Date	Price
2026-08-21	309.35
2026-08-20	311.30
2026-08-19	316.83
2026-08-18	310.03
2026-08-17	305.59

Date	Price
2026-07-30	333.43
2026-07-29	338.19
2026-07-28	340.08
2026-07-27	336.91
2026-07-24	333.02

```
# Normalize the data
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df['Price'].values.reshape(-1, 1))

# Print the length of the scaled data for clarity
print(f"Length of scaled data: {len(scaled_data)} rows")

# Create sequences for LSTM
def create_sequences(data, time_step):
    X, y = [], []
    for i in range(len(data) - time_step - 1):
        a = data[i:(i + time_step), 0]
        X.append(a)
        y.append(data[i + time_step, 0])
    return np.array(X), np.array(y)

# Adjust time_step based on data length. For demonstration, we'll use a small value.
# In a real scenario, you'd want significantly more data than time_step + 1.
# If len(scaled_data) is 21, and time_step = 5, then 21 - 5 - 1 = 15 iterations. This should work.
time_step = min(5, len(scaled_data) - 2) # Ensure time_step is at least 1, and fits the data
if time_step < 1:
    raise ValueError("Not enough data to create sequences. Need at least 3 data points.")

X, y = create_sequences(scaled_data, time_step)

# Check if X is empty after sequence creation
if X.size == 0:
    raise ValueError(f"No sequences could be created with time_step={time_step}. \n" +
                     f"Scaled data length: {len(scaled_data)}. \n" +
                     f"You need at least {time_step + 2} data points to create sequences.")

# Reshape X for LSTM input: [samples, time_steps, features]
X = X.reshape(X.shape[0], X.shape[1], 1)

print(f"Using time_step: {time_step}")
print(f"Shape of X (input sequences): {X.shape}")
print(f"Shape of y (target values): {y.shape}")
```

```
Length of scaled data: 21 rows
Using time_step: 5
```

```
Shape of X (input sequences): (15, 5, 1)
Shape of y (target values): (15,)
```

```
# Split data into training and testing
training_size = int(len(X) * 0.8)
test_size = len(X) - training_size
X_train, X_test = X[0:training_size,:]
y_train, y_test = y[0:training_size],

print(f"X_train shape: {X_train.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (12, 5, 1)
y_train shape: (12,)
X_test shape: (3, 5, 1)
y_test shape: (3,)
```

```
# Build the LSTM model
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(time_step, 1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(units=1))

model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 5, 50)	10,400
dropout (Dropout)	(None, 5, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

Total params: 30,651 (119.73 KB)
Trainable params: 30,651 (119.73 KB)
Non-trainable params: 0 (0.00 B)

```
# Train the model
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.1, verbose=1)

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



```
Epoch 1/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step - loss: 0.0312 - val_loss: 0.3496
Epoch 2/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 320ms/step - loss: 0.0276 - val_loss: 0.3356
Epoch 3/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 209ms/step - loss: 0.0243 - val_loss: 0.3220
Epoch 4/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 161ms/step - loss: 0.0210 - val_loss: 0.3087
Epoch 5/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 104ms/step - loss: 0.0186 - val_loss: 0.2955
Epoch 6/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step - loss: 0.0166 - val_loss: 0.2824
Epoch 7/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 101ms/step - loss: 0.0146 - val_loss: 0.2696
Epoch 8/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 102ms/step - loss: 0.0128 - val_loss: 0.2570
Epoch 9/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step - loss: 0.0124 - val_loss: 0.2452
Epoch 10/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step - loss: 0.0107 - val_loss: 0.2341
Epoch 11/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 107ms/step - loss: 0.0113 - val_loss: 0.2247
Epoch 12/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step - loss: 0.0107 - val_loss: 0.2169
Epoch 13/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 103ms/step - loss: 0.0110 - val_loss: 0.2114
Epoch 14/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 113ms/step - loss: 0.0135 - val_loss: 0.2085
Epoch 15/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 96ms/step - loss: 0.0129 - val_loss: 0.2079
Epoch 16/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step - loss: 0.0123 - val_loss: 0.2089
Epoch 17/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 109ms/step - loss: 0.0118 - val_loss: 0.2114
Epoch 18/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 143ms/step - loss: 0.0125 - val_loss: 0.2148
Epoch 19/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step - loss: 0.0131 - val_loss: 0.2189
Epoch 20/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 112ms/step - loss: 0.0097 - val_loss: 0.2228
```




```
# Make predictions
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

# Inverse transform the predictions an
train_predict = scaler.inverse_transfo
y_train_inverse = scaler.inverse_trans
test_predict = scaler.inverse_transfor
y_test_inverse = scaler.inverse_transf

# Prepare data for plotting
look_back = time_step

# Shift train predictions for plotting
train_plot = np.empty_like(scaled_data
train_plot[:, :] = np.nan
train_plot[look_back:len(train_predict

# Shift test predictions for plotting
test_plot = np.empty_like(scaled_data)
test_plot[:, :] = np.nan
# Corrected indexing for test_plot
test_plot[len(train_predict) + look_ba
```

```
# Plot actual vs. predicted values
plt.figure(figsize=(15, 8))
plt.plot(scaler.inverse_transform(scal
plt.plot(train_plot, label='Train Prec
plt.plot(test_plot, label='Test Predic
plt.title('Stock Price Prediction usir
plt.xlabel('Time (Days)')
plt.ylabel('Stock Price')
plt.legend()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 373ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```

